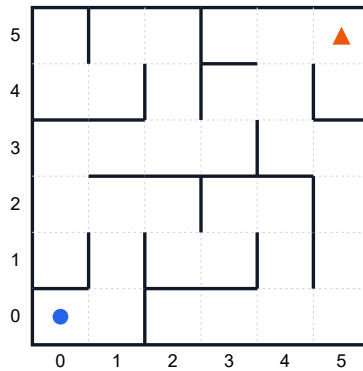


[프라임칼리지] 인공지능시스템 - 1차 과제물

이름: 최종찬

다음 미로의 입구(●, (0, 0) 위치)에서 출발하여 출구(▲, (5, 5) 위치)로 나오는 경로를 탐색하려고 한다.
이동은 상, 하, 좌, 우의 방향으로 1칸씩 할 수 있다고 가정한다.



특정 좌표에서 출구까지의 거리의 예측은 각 축의 거리의 합으로 한다.
예를 들어 (2, 4)의 거리는 다음과 같이 예측한다.

$$\text{예측 거리} = (5 - 2) + (5 - 4) = 4$$

(가) 상태공간 탐색으로 이 문제를 풀이할 수 있도록 이 문제를 표현하라.

상태묘사

현재 위치한 칸의 좌표를 두 정수 x, y 로 이루어진 객체로 표현한다.

```
interface State {  
  x: number;  
  y: number;  
}
```

초기상태와 목표상태의 정의

초기상태는 입구 (0, 0), 목표상태는 출구 (5, 5)이다.

현재 좌표가 목표 좌표와 같으면 목표에 도달한 것으로 판단한다.

```
const initial: State = { x: 0, y: 0 };  
const goal: State = { x: 5, y: 5 };  
function isGoal(s: State): boolean {  
  return s.x === goal.x && s.y === goal.y;  
}
```

연산자

상하좌우로 한 칸 이동한다.

미로 밖으로 나가거나 벽을 통과하는 이동은 제외한다.

```
interface Operator {  
  dx: number;  
  dy: number;  
}  
  
const operators: Operator[] = [  
  { dx: 0, dy: 1 }, { dx: 1, dy: 0 },  
  { dx: 0, dy: -1 }, { dx: -1, dy: 0 },  
];
```

// verticalWalls[x][y]: (x,y)와 (x+1,y) 사이의 벽 유무

```
const verticalWalls: boolean[][] = [  
  [false, true, false, false, false, true ], // x=0  
  [true, true, false, false, true, false], // x=1  
  [false, false, true, false, true, true ], // x=2  
  [false, true, false, true, false, false], // x=3  
  [false, true, true, false, true, false], // x=4  
];
```

// horizontalWalls[y][x]: (x,y)와 (x,y+1) 사이의 벽 유무

```
const horizontalWalls: boolean[][] = [  
  [true, false, true, true, false, false], // y=0  
  [false, false, false, false, false, false], // y=1  
  [false, true, true, true, true, false], // y=2  
  [true, true, false, false, false, true ], // y=3  
  [false, false, false, true, false, false], // y=4  
];
```

// 연산자: 상, 우, 하, 좌의 이동 중 가능한 후계 상태를 생성한다.

```
function successors({ x, y }: State): State[] {  
  const result: State[] = [];  
  for (const { dx, dy } of operators) {  
    const nx = x + dx, ny = y + dy;  
    if (nx < 0 || nx > 5 || ny < 0 || ny > 5) continue;  
    if (dx && verticalWalls[Math.min(x, nx)][y]) continue;  
    if (dy && horizontalWalls[Math.min(y, ny)][x]) continue;  
    result.push({ x: nx, y: ny });  
  }  
  return result;  
}
```

상태공간

초기상태에서 연산자를 반복 적용하여 도달할 수 있는 모든 상태의 집합이다.
같은 좌표는 한 번만 저장하며, 이 미로에서는 총 36개 상태가 생성된다.

```
const stateSpace = new Map<string, State>();  
const pending: State[] = [initial];  
for (const state of pending) {  
  const key = `${state.x},${state.y}`;  
  if (stateSpace.has(key)) continue;  
  stateSpace.set(key, state);  
  pending.push(...successors(state));  
}  
// 초기상태부터의 유효한 경로에서 각 이동 비용은 1이다.  
function pathCost(path: State[]): number {  
  return path.length - 1;  
}
```

successors는 현재 위치에서 한 번의 이동으로 도달할 수 있는 후계 상태들을 반환한다.
해는 초기상태에서 목표상태까지의 유효한 경로이며, 이동 횟수가 가장 적은 경로가 최적해이다.

(나) 언덕오르기 탐색을 적용하여 문제의 해를 구하려고 한다.
 평가함수를 정의하고, 이에 따른 탐색 트리를 구하라.
 트리의 노드에는 확장 순서 및 경로비용을 표시하라.

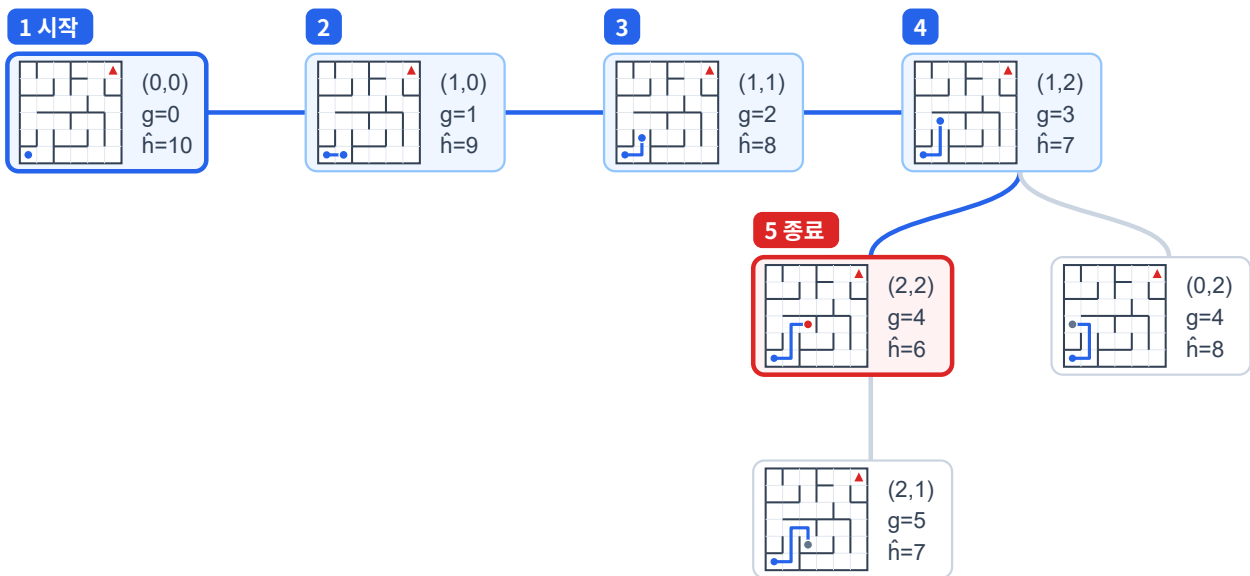
평가함수: 현재 위치에서 출구까지의 가로거리와 세로거리를 더한 값을 사용한다.
 미로의 가로폭을 W , 세로폭을 H 라 하면, 이 문제에서는 $W = 6, H = 6$ 이다.
 좌표는 0부터 시작하므로 출구의 좌표는 $(W - 1, H - 1)$ 이다.
 현재 위치 (x, y) 에서의 평가함수는 다음과 같다.

$$\hat{f}(n) = \hat{h}(n) = (W - 1 - x) + (H - 1 - y).$$

평가값이 작을수록 출구에 가까운 상태로 본다.

선택 기준: 언덕오르기 탐색은 현재 상태의 후계 상태들을 출구까지의 예상 비용 $\hat{h}(n)$ 으로 비교하여 다음 상태를 선택하는 접근이다.
 이미 이동한 경로 비용 $g(n)$ 은 선택 기준에 포함되지 않는다.
 더 작은 값을 갖는 후계 상태가 없으면 멈춘다.

탐색 결과: $(0, 0) \rightarrow (1, 0) \rightarrow (1, 1) \rightarrow (1, 2) \rightarrow (2, 2)$ 로 이동하며 $\hat{h}$ 는 10, 9, 8, 7, 6으로 감소한다.
 하지만 $(2, 2)$ 의 이웃은 $(1, 2)$ 와 $(2, 1)$ 뿐이고, 둘 다 $\hat{h} = 7$ 이다.
 따라서 $(2, 2)$ 는 이 평가함수의 지역최소점이며, 출구에 도달하지 못하고 경로비용 4에서 탐색이 종료된다.



(다) A* 알고리즘을 적용하여 최단길이 경로를 구하려고 한다.

평가함수를 정의하고, 이에 따른 탐색 트리를 구하라.

트리의 노드에는 확장 순서 및 평가함수의 값을 표시하라.

정의한 평가함수를 사용할 경우 최단길이 경로를 탐색할 수 있는지 설명하라.

평가함수: 출발점부터의 경로 비용과 출구까지의 예상 비용을 더한다.

예상 비용 $\hat{h}(n)$ 은 (나)의 언덕오르기 탐색과 동일하게 가로거리와 세로거리를 더한 값을 사용한다.

$$\hat{f}(n) = g(n) + \hat{h}(n), \quad \hat{h}(n) = (W - 1 - x) + (H - 1 - y).$$

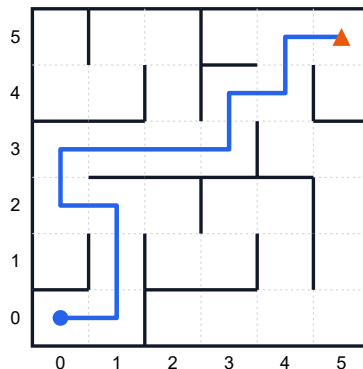
선택 기준: OPEN 전체에서 $\hat{f}$ 가 최소인 노드를 선택하므로, 현재 상태에서 $\hat{h}$ 가 증가하는 이동도 후보로 유지할 수 있다. $\hat{f}$ 가 같으면 OPEN에 먼저 들어온 노드를 선택한다.

같은 상태가 중복 생성되면 더 작은 g 만 유지하고, 같거나 큰 비용으로 재방문하는 경로는 제거한다.

출구를 생성한 순간이 아니라 OPEN에서 출구를 선택했을 때 종료한다.

탐색 결과: 출구 (5, 5)를 20번째로 선택하며, 이때 $g = 12$, $\hat{h} = 0$, $\hat{f} = 12$ 이다.

출구에 도달하여 경로비용 12에서 탐색이 종료된다.



정의한 평가함수로 최단길이 경로를 탐색할 수 있는 이유:

노드 n 으로부터 목표노드까지의 경로비용 예측치 $\hat{h}(n)$ 이 항상 실제 경로비용 $h(n)$ 이하라면, A* 알고리즘은 최소비용 경로를 탐색하는 것을 보장한다.

$$\hat{h}(n) \leq h(n).$$

이 문제에서는 현재 좌표 (x, y) 에서 출구까지의 가로거리와 세로거리의 합을 $\hat{h}(n)$ 으로 사용했다.

목표상태에 도달하려면 가로로 최소한 $W - 1 - x$ 칸, 세로로 최소한 $H - 1 - y$ 칸을 이동해야 한다.

벽으로 인해 우회하는 경우에는 이동 횟수가 더 늘어날 수 있지만, 이 거리의 합보다 적은 횟수로 목표상태에 도달할 수는 없다. 따라서 모든 노드에서 $\hat{h}(n) \leq h(n)$ 이 성립한다.

OPEN에서 목표노드를 선택한 경우 나머지 노드들의 평가함수 값은 선택한 목표노드까지의 경로비용보다 작을 수 없다.

또한 경로비용 예측치가 실제 비용 이하이므로, 그 노드들을 거쳐 목표노드에 도달하는 실제 경로비용도 탐색된 경로의 비용보다 작을 수 없다.

그러므로 앞서 정의한 평가함수를 사용하여 탐색한 경로는 최단길이 경로다.

